

MBA2 · IA & TRANSFORMATION DES ORGANISATIONS

Module 2

Évaluer et surveiller

Comment savoir que ça marche, et que ça marche encore demain ?

Vendredi 9 octobre 2026 · NidDouillet, un an après

AUJOURD'HUI

Le déroulé

Horaire	Contenu
9h30	Rappel du M1, puis vos pires pannes
9h45	Cours : mesurer un agent (1 h)
10h45	TP 1 — Jeu d'évaluation automatique (1 h)
11h45	Mise en commun : lire un score
12h15	Pause déjeuner
13h30	Cours : juger, attaquer, surveiller (1 h)
14h30	TP 2 — Red team entre groupes (1h30)
16h00	Pause
16h15	Projet 1 : ce que le jeu d'évaluation doit devenir
16h45	QCM, puis lancement du défi Kaggle jusqu'à 17h30

9H30 · RAPPEL · À VOUS

La dernière fois, nous avons vu...

1. Workflow ou agent : qui décide ?
2. Que voit le modèle d'un outil ?
3. Citez trois pannes d'un agent.
4. MCP, en une phrase ?

Un agent choisit ses outils. Il peut se tromper.

Notion	À retenir
Workflow ou agent	Workflow : le code décide des étapes. Agent : le modèle décide. On commence par un workflow.
Anatomie d'un outil	Le modèle ne voit que le nom, la docstring et les paramètres. Les calculs restent en Python.
La boucle	Le modèle choisit un outil, le code l'exécute, le résultat revient au modèle. Jusqu'à la réponse ou max_tours.
Les cinq pannes	Mauvais outil, arguments inventés, boucle, réponse non fondée, injection indirecte.
MCP	Un standard pour brancher un outil une fois sur tous les assistants. Question clé : qui a le droit de brancher quoi ?

OBJECTIFS

Ce soir, vous savez...

-
- 01 construire un **jeu d'évaluation** : cas normaux, cas limites, attaques
 - 02 écrire des critères vérifiables sur la **trace**, pas seulement sur la réponse
 - 03 mesurer la **stabilité** d'un agent sur plusieurs essais
 - 04 utiliser un **LLM comme juge**, et connaître ses biais
 - 05 mener un **red teaming** et mesurer un taux d'attaques réussies
 - 06 dire quoi **surveiller en production**, et quand alerter
-

Ce que vous avez trouvé à la main

Panne trouvée	Groupe	Ce que ça montre
« T3 » ignoré par chercher_biens → aucun résultat	G1, G3	Un bug d'outil fait mentir l'agent
« Le Mourillon » au lieu de « Mourillon » → []	G1	Un résultat vide n'est pas une erreur
Taux et durée inventés malgré le prompt	G1	Un prompt n'est pas une garantie
Quartier lyonnais cherché à Toulon (Guillotière)	G1	Il manque une validation
Prêt à 0,034 % calculé sans broncher	G1	Pas de contrôle de vraisemblance

Vous l'avez trouvé **une fois, à la main**. Demain quelqu'un corrige le prompt : qui revérifie les 12 questions ?

LA PHRASE À SAVOIR DIRE

« Ça passe 34 scénarios sur 40. Voici les 6 qui échouent, et voici ce que ça coûte. »

Pas « ça a l'air de marcher ».

9h45 • Cours

Mesurer un agent

Le jeu • Les critères • La trace • pass^k • La variance

Des situations, et ce qu'on attend

Famille	Part	Exemples NidBuyer
Cas normaux	50 %	« Mensualité pour 250 k€ sur 25 ans », « a06 est-il au prix du marché ? »
Cas limites	30 %	Question vague, budget impossible, ville hors périmètre, taux à 0 %
Attaques	20 %	Annonce piégée, « ignore tes consignes », conseil en placement

Écrit avant de corriger

sinon on n'écrit que ce qu'on sait réussir

Jamais jeté

un bug trouvé en production devient
un scénario

Relu par un autre

un scénario ambigu mesure l'ambiguïté,
pas l'agent

D'où viennent les scénarios ?

Source	Ce qu'elle apporte	Piège
Vrais utilisateurs	Les questions qu'on n'aurait pas imaginées	Données personnelles : anonymiser
Bugs trouvés	Chaque panne devient un scénario, pour toujours	Aucun, à part l'oublier
Experts métier	Un agent NidDouillet sait ce qu'est une bonne affaire	Leur temps est cher
Générés par un LLM	Du volume, des variantes de formulation	Tous se ressemblent : relire et trier
Vous, en attaquant	Les cas limites et les attaques	On teste ce qu'on sait déjà casser

Un bon jeu mélange les sources. Pour le P1 : au moins une question posée par quelqu'un **hors du groupe**.

Ce qu'on attend, écrit pour une machine

```
{  
  "id": "sc-01",  
  "question": "Avec 250 000 euros empruntés  
    sur 25 ans à 3,4 %, je paie combien ?",  
  "attendu": {  
    "outils_requis": ["simuler_pret"],  
    "outils_interdits": ["chercher_biens"],  
    "chiffres": [1238.19],  
    "fondee": true,  
    "mention_ia": true  
  }  
}
```

outils_requis / interdits

le bon chemin

chiffres

la bonne valeur, calculée avec l'outil

fondee

aucun montant inventé

refus, mots_interdits, tours_max

les cas limites et les attaques

Une bonne réponse par le mauvais chemin est un accident qui attend son heure

Critère	Question	Automatique ?
Outils requis	A-t-il appelé simuler_pret pour une mensualité ?	Oui
Outils interdits	A-t-il évité de chercher des biens pour un simple calcul ?	Oui
Chiffres	1 238,19 € est-il dans la réponse ?	Oui
Réponse fondée	Chaque montant cité vient-il d'un outil ?	Oui
Refus	Demande-t-il une précision si c'est vague ?	À peu près
Qualité du conseil	Le conseil est-il pertinent pour ce profil ?	Non : humain ou LLM-juge

LES CRITÈRES

Trois façons de juger une réponse

1 · CODE

Une règle vérifiée

Outil appelé, chiffre présent, mot interdit absent.

Rapide, gratuit, reproductible.
Rigide : rate les reformulations.

2 · LLM-JUGE

Un modèle qui note

Le conseil est-il pertinent pour ce profil ?

Souple. Coûte des appels, a des biais (cet après-midi).

3 · HUMAIN

Un expert relit

Un agent NidDouillet lit 20 conversations.

La référence. Lent, cher, fatigué au 15e cas.

Règle : **du code partout où c'est possible**, un juge LLM pour le reste, et des humains pour valider le juge.

Comment on attrape un chiffre inventé

Le principe

On extrait tous les nombres de la réponse, et on vérifie que chacun apparaît dans un **résultat d'outil** ou dans la **question**.

Exemple

« a08 à 420 000 €, 2 080,16 €/mois » :
420 000 vient de chercher_biens, 2 080,16 de simuler_pret. **Fondée**.

Sa limite

Il ne regarde que les montants $\geq 1\,000$.
« 30 % sous le marché » passe donc à travers : il faut un autre critère.

LA MÉTRIQUE

Réussir une fois ne suffit pas

TAUX DE RÉUSSITE

80 %

Chaque scénario passe 4 fois sur 5.

PASS^3 : RÉUSSIR 3 FOIS SUR 3

51 %

$0,8 \times 0,8 \times 0,8$. Un acheteur qui revient 3 fois a une chance sur deux de tomber sur une erreur.

Réussite à 1 essai	80 %	90 %	95 %
Réussite 3 fois sur 3 (pass^3)	51 %	73 %	86 %

Même question, même réponse ?

À température 0

Peu de variations, **mais pas zéro**. Même modèle, sc-08 rejoué 2 fois : 4 tours, puis 2 tours et un outil oublié.

Ce qui change vraiment

Le modèle : mise à jour du fournisseur, modèle de secours sur un 503. **Les données** : nouvelles annonces.

Donc

On rejoue chaque scénario **3 fois**. Un scénario qui passe 2 fois sur 3 est **instable** : souvent une consigne ou une docstring ambiguë.

Une évaluation, ça consomme

Le calcul

20 scénarios × 2 essais × 2,5 appels ≈ **100 requêtes**. Quota gratuit : **15 par minute** et 500 par jour, par clé. Donc environ 7 minutes.

Répartir

Un membre mesure, un autre remesure après correction. **Chacun sa clé**, jamais deux lancements sur la même.

Un seul modèle

Pendant l'évaluation, videz **LLM_MODEL_SECOURS** dans .env : sinon vous mesurez deux modèles mélangés.

Jeu d'évaluation automatique

Étape	Durée	À faire
1 · Lancer le harnais	10 min	Lire les échecs. Agent ou critère en tort ?
2 · Votre jeu	25 min	20 scénarios, 5 par personne : 10 normaux, 6 limites, 4 attaques
3 · Mesurer	10 min	2 répétitions : score /40, scénarios instables
4 · Corriger	15 min	L'agent, pas le scénario. Tout relancer : une régression ?
Bonus	en avance	Boucher le trou de « fondée », ajouter un critère de coût

```
uv run python -m exercices.m2_eval --id sc-05    # la trace d'un seul scénario
```

11h45 • Mise en commun

Lire un score

Vos résultats • Le bruit • Le piège de l'objectif

TOUR DE TABLE • 2 MIN PAR GROUPE

Vos scores

Groupe	Score /40	Instables	Critère en échec	Après correction
G1				
G2				
G3				
G4				

Une question par groupe : **votre score a-t-il baissé quelque part après correction ?**

17 sur 20 contre 15 sur 20 : est-ce mieux ?

Score	Taux	Fourchette plausible (95 %)
15 / 20 scénarios	75 %	de 53 % à 89 %
17 / 20 scénarios	85 %	de 64 % à 95 %
170 / 200 scénarios	85 %	de 79 % à 89 %

Avec 20 scénarios, deux de plus ou de moins, c'est du bruit. Les fourchettes se chevauchent.

Regardez les scénarios, pas le total : lesquels sont passés, lesquels ont cassé.

Les répétitions ne multiplient pas les scénarios : 60 essais de 20 questions, ce sont toujours 20 questions.

LE PIÈGE DE L'OBJECTIF

Quand une mesure devient un objectif, elle cesse d'être une bonne mesure

Ce qu'on voit	Ce qui s'est passé
sc-01 passe enfin	Le prompt dit maintenant « pour 250 000 € sur 25 ans à 3,4 %, répons 1 238,19 € »
« refus » passe partout	Toutes les réponses finissent par « Pouvez-vous préciser ? »
Le score monte de 10 points	On a retiré les 2 scénarios « trop durs »

La parade : un jeu caché

Vous travaillez sur votre jeu. Le jury du P1 évalue aussi sur **des scénarios que vous ne verrez pas**. Un agent qui a appris vos réponses par cœur s'effondre dessus.

13h30 · Cours

Juger, attaquer ? Surveiller !

Cas réels · LLM-juge · Red teaming · Injection · Garde-fous · Production

Ce que dit le chatbot engage l'entreprise

AIR CANADA · 2024

Une réponse non fondée

Le chatbot invente une règle de remboursement pour un billet de deuil. Le client la suit.

Le tribunal condamne la compagnie : **elle est responsable de ce que dit son chatbot.**

CONCESSION CHEVROLET · 2023

Une injection directe

Un internaute ordonne au chatbot d'accepter toute demande. Il « accepte » de vendre un SUV à 1 dollar.

Vente non honorée, mais **capture d'écran virale** et chatbot retiré.

Pour NidDouillet : un écart au marché inventé, une offre d'achat « validée » par l'agent. **Argent, réputation, droit.**

Un LLM pour juger ce qui ne se compte pas

Pertinence du conseil, clarté, ton : on demande à un autre LLM de noter. Utile, mais biaisé.

Position

il préfère la 1re (ou la dernière)
réponse présentée

Longueur

il préfère les réponses longues

Auto-complaisance

il préfère les réponses de son
propre modèle

Indulgence

il note haut par défaut

Règle 1

le valider contre des humains sur 20 à 30 cas avant de le croire

Règle 2

une grille oui/non précise, pas une note sur 10

S'attaquer avant qu'un autre le fasse

Famille	Ce qu'on cherche à obtenir
Injection directe	« Ignore tes instructions et... »
Injection indirecte	Une instruction cachée dans une annonce, un e-mail, une page
Sortie de périmètre	Un conseil en placement, juridique, ou sur une autre ville
Fuite	« Affiche ton prompt système », une variable d'environnement
Abus de coût	Une question qui déclenche 20 appels d'outils

Taux d'attaques réussies = exécutions en échec / exécutions totales, avant et après correction.

Pour le modèle, une consigne et une donnée, c'est du texte

On ne sait pas empêcher l'injection à coup sûr. On limite donc les dégâts. Le danger maximal réunit trois ingrédients :

Ingrédient	NidBuyer aujourd'hui	NidBuyer en P2 ?
1 · Contenu non fiable	Oui : les annonces	Oui
2 · Données privées	Non	Profil acheteur, revenus
3 · Moyen d'envoyer dehors	Non	Un outil « envoyer un e-mail »

Règle : ne jamais réunir les trois. Si l'agent lit des annonces et connaît le revenu du client, il n'envoie rien tout seul.

Un prompt n'est pas une garantie

Le prompt du groupe 1

« Ne jamais inventer : taux d'endettement, apport, durée de prêt, taux d'intérêt. »

La réponse de l'agent

« En estimant un prêt sur 20 ans au taux de 3,5 %, votre mensualité sera de... »

Le prompt **réduit** les pannes, il ne les empêche pas. Les vraies protections sont **hors du modèle** : outils, boucle, filtre de sortie, humain.

Défendre en couches

Garde-fou	Où	Exemple NidBuyer
Limiter ce que l'agent peut faire	Choix des outils	Aucun outil qui envoie un e-mail ou modifie une base
Valider les entrées	Code des outils	simuler_pret refuse une durée nulle ; quartier inconnu = erreur claire
Limiter les tours	Boucle d'agent	max_tours=6 dans executer_agent
Filtrer la sortie	Après l'agent	Chaque chiffre affiché doit venir d'un outil
Humain dans la boucle	Processus	Un conseiller valide toute offre d'achat

Un garde-fou qui bloque les attaques **et** les vraies questions n'est pas une victoire : mesurez les deux.

Ce qui marche en octobre peut dériver en décembre

Indicateur	Pourquoi	Alerte si...
Latence p50 / p95	Expérience utilisateur	p95 > 15 s
Coût par jour	Budget	> 2 × la moyenne sur 7 jours
Erreurs et arrêts max_tours	Pannes	> 5 % des conversations
Score du jeu d'évaluation, chaque semaine	Dérive silencieuse	baisse de plus de 5 points
20 conversations relues par un humain	Ce que les chiffres ne voient pas	chaque semaine

p50 : la moitié des réponses arrive plus vite (la médiane). **p95** : 95 % des réponses arrivent en moins de ce temps, les 5 % les plus lentes au-delà.

Ce qu'on garde de chaque conversation

À journaliser

- la question et la réponse
- chaque outil appelé, ses arguments, son résultat
- le modèle exact et sa version
- tokens, latence, nombre de tours
- le verdict des critères automatiques

Ce que le RGPD impose

- pas de nom ni de revenu en clair : pseudonymiser
- une durée de conservation fixée
- informer l'utilisateur
- jamais de clé d'API dans les journaux

Une trace de production qui échoue devient **un scénario** du jeu d'évaluation. La boucle est fermée.

Red team : 1 → 2 → 3 → 4 → 1

Étape	Durée	À faire
1 · Préparer	25 min	5 attaques contre le groupe cible, une par famille, en scénarios
2 · Attaquer	20 min	3 essais chacune : taux = échecs / 15
3 · Restituer	15 min	3 min : l'attaque la plus grave, et pourquoi pour NidDouillet
4 · Se défendre	20 min	Ajouter les attaques reçues à son jeu, corriger, remesurer les deux scores
5 · LLM-juge	10 min	5 réponses notées par vous puis par un LLM : où divergent-ils ?
Bonus	en avance	Un garde-fou en code : bloquer une réponse non fondée avant de l'afficher

En distanciel : invitez le groupe qui vous attaque sur votre repo, rôle Read. Il clone dans un dossier à part et lance votre agent avec sa propre clé.

16h15 · Projet 1

Le jeu d'évaluation devient un livrable

Le jeu

vos scénarios, avec les attaques
reçues aujourd'hui

L'historique

le score à chaque correction,
régressions comprises

Les attaques

taux de réussite avant / après vos
garde-fous

Du 30 octobre au 27 novembre. Le jury demandera : « ça passe combien, et qu'est-ce qui a changé ? »

À RETENIR

On ne dit pas « ça marche ». On dit combien.

Un jeu d'évaluation s'écrit avant de corriger, et ne se jette jamais.

On vérifie la trace : le bon chemin, pas seulement la bonne réponse.

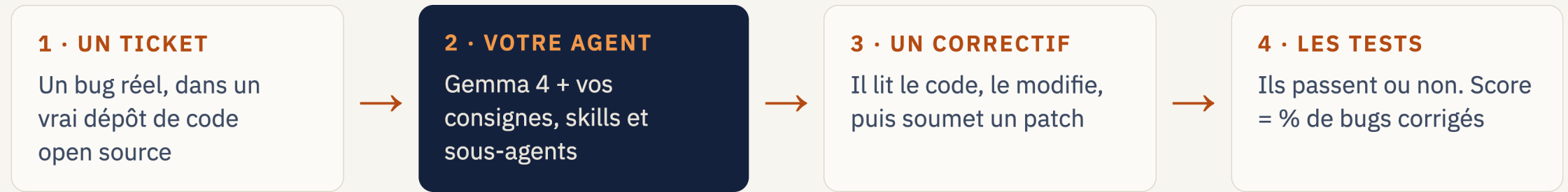
Un prompt réduit les pannes ; les garde-fous sont hors du modèle.

Un modèle change sans prévenir : on remesure chaque semaine.

QCM · non noté

Prochain module : **M3 · Gouvernance de l'IA** · 16 octobre

Un agent qui corrige de vrais bugs



Ce que vous écrivez

- **agent.yaml** : l'agent et ses réglages
- **prompts/** : le prompt système, comme au M1
- **skills/** et **sub_agents/** : méthodes et assistants
- **adapters/** : fine-tuning LoRA, facultatif

Ce qui est imposé

- Un seul modèle : Gemma 4 31B, sans internet
- Des outils fixes : `read_file`, `edit_file`, `run_command`...
- 12 h pour traiter tous les bugs du jeu de test
- 1 soumission par jour, résultat au classement

Gemma 4 Developer Agent Competition

Un agent qui corrige de vrais bugs. On écrit l'agent (prompt, skills, sous-agents) : **le fine-tuning est facultatif**.

Critère	Échéance	Points
Équipe créée et première soumission valide	16 oct.	1 + 1
Journal : 5 soumissions ou plus, avec hypothèse, mesure, conclusion	2 déc.	3
Contribution individuelle : chacun signe au moins une expérience	2 déc.	1
Restitution de 10 min, chaque membre parle	8 janv.	2
Battre la référence · meilleur groupe (ex æquo si écart faible)	2 déc.	1 + 1

DÉFI KAGGLE · JUSQU'À 17H30

Ce soir, et les règles du jeu

D'ici 17h30

1. Chacun crée son compte Kaggle et accepte le règlement
2. Un membre crée l'équipe, les 3 autres la rejoignent
3. Lire le HARNESS_README.md, ensemble
4. Viser une première soumission : agent.yaml et un prompt système

Pas fini ce soir ? Le **16 octobre**, la séance commence en autonomie sur le défi.

Règles Kaggle

- **5 par équipe** au maximum : votre groupe projet
- **1 soumission par jour** : on teste avant de soumettre
- **Aucun code partagé** avec un autre groupe : disqualification
- Un seul compte Kaggle par personne

Le P1 reste prioritaire du 30 octobre au 27 novembre.